



Università di Genova

High Performance Computing Report CUDA

Andrea Cattarinich 5137057

November 26, 2025

Contents

1	Architectures	2
1.1	HP Envy 17-ae103nl (Laptop)	2
1.2	Google Colab	2
2	Hotspot Analysis	3
2.1	Code Description	4
2.2	Project Structure	7
2.3	Profiling	7
3	Parallelization	9
3.1	Sequential section	9
3.2	CUDA	9
3.2.1	Code	9
3.2.2	Compilation	10
3.2.3	Execution	10
3.2.4	Results	10
4	Conclusions	11
4.1	Realistic System Simulation	11
4.2	Considerations	11

1 Architectures

For this project, I used two workstations: my personal laptop and the Google Colab environment

1.1 HP Envy 17-ae103nl (Laptop)

CPU

- **Model:** Intel Core i7-6500U @ 2.50 GHz
- **Architecture:** x86_64
- **Physical cores:** 2
- **Logical threads:** 4 (Hyper-Threading)

GPU

- **Model:** NVIDIA GeForce GTX 950M
- **CUDA Compute Capability:** 5.0
- **Driver version:** 572.16
- **CUDA Version:** 12.8 (*CUDA Toolkit 12.x*)

Software and Setup:

- Visual Studio Community 2022 - Workload: Desktop development with C++
- CUDA Toolkit installed manually to enable GPU programming
- WSL2
- NVIDIA drivers (version 572.16) installed for WSL2 from NVIDIA official website

1.2 Google Colab

CPU

- **Model:** Intel(R) Xeon(R) CPU @ 2.20GHz
- **Architecture:** x86_64
- **Physical cores:** 1
- **Logical threads:** 2 (Hyper-Threading)
- **Socket(s):** 1

GPU

- **Model:** NVIDIA Tesla T4
- **CUDA Compute Capability:** 7.5
- **Driver version:** 550.54.15
- **CUDA Version:** 12.8 (*CUDA Toolkit 12.x*)

2 Hotspot Analysis

Heat conduction (or heat diffusion) is the process by which thermal energy spreads within a material due to temperature differences. In two dimensions, this phenomenon can be represented as heat spreading across a plate or surface over time. Numerical simulations of heat conduction are widely used in engineering, physics, and high-performance computing to predict temperature distribution and analyze thermal effects.

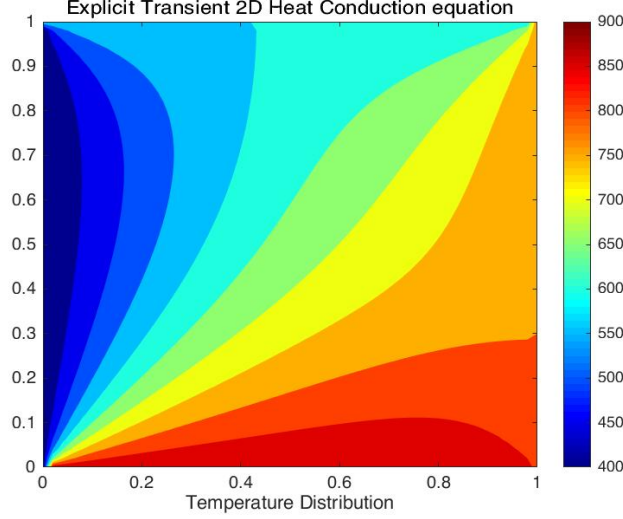


Figure 1: Illustration of 2D heat conduction simulation.

The formula to represent 2D heat conduction is given as follows:

$$\frac{\partial T}{\partial t} = \alpha \left(\frac{\partial^2 T}{\partial x^2} + \frac{\partial^2 T}{\partial y^2} \right) \quad (1)$$

where:

- T is temperature;
- t is time;
- α is the thermal diffusivity;
- (x, y) are points in a grid.

To solve this problem numerically, one possible finite difference approximation is:

$$\frac{\Delta T}{\Delta t} = \frac{T_{i,j}^{n+1} - T_{i,j}^n}{\Delta t} = \alpha \left(\frac{T_{i+1,j}^n - 2T_{i,j}^n + T_{i-1,j}^n}{(\Delta x)^2} + \frac{T_{i,j+1}^n - 2T_{i,j}^n + T_{i,j-1}^n}{(\Delta y)^2} \right) \quad (2)$$

where:

- Δt is the timestep;
- i, j are indices in the grid;
- n indicates the current timestep, so $T_{i,j}^n$ is the temperature at point (i, j) at time $t = n\Delta t$, and $T_{i,j}^{n+1}$ is the temperature at the next timestep $t = (n+1)\Delta t$.

2.1 Code Description

This program simulates the diffusion of heat on a two-dimensional plate using the finite difference method. It includes two main implementations:

1. **Reference CPU version** - `step_kernel_ref()` - serial implementation
2. **Modified version** - `step_kernel_mod()` - intended for CUDA acceleration

Both implementations are designed to produce the same results.

Data Layout

The simulation uses a grid of size `ni × nj` (columns × rows).

Temperature values are stored in a **1D array**. A macro is used to convert 2D indices to linear index:

```
1 #define I2D(num, c, r) ((r)*(num)+(c))
```

Initialization

In the beginning, the boundary points of the grid are assigned with initial *random* temperatures, while the inner points update their temperature iteratively according to the formula above.

```
1 for (int i = 0; i < ni * nj; ++i) {  
2     t1_ref[i] = t2_ref[i] =  
3     t1[i]     = t2[i]     =  
4         (float)rand() / (float)(RAND_MAX / 100.0f);  
5 }
```

Realistic System Inizialization

In addition to the synthetic configurations used, I also decided to simulate a more realistic scenario in order to visually inspect the physical behavior of the system. Specifically, I initialized the domain by placing a high-temperature square region at the center of the grid, representing a localized heat source. All remaining points were initialized with a low random temperature, mimicking environmental thermal noise. The initialization code is reported below:

```
1 // Hot square parameters  
2 int hot_size = 40;      // side length of the hot square  
3 int hot_temp = 80;      // temperature of the hot region  
4 int hot_i0 = ni/2 - hot_size/2;    // center coordinate in i  
5 int hot_j0 = nj/2 - hot_size/2;    // center coordinate in j  
6  
7 for(int j = 0; j < nj; j++){  
8     for(int i = 0; i < ni; i++){  
9         int idx = I2D(ni, i, j);  
10  
11         if(i >= hot_i0 && i < hot_i0 + hot_size &&
```

```

12         j >= hot_j0 && j < hot_j0 + hot_size)
13     {
14         // heat source
15         temp1[idx] = temp2[idx] = (float)hot_temp;
16     }
17     else
18     {
19         // random background
20         temp1[idx] = temp2[idx] = (float)rand()/RAND_MAX*30.0f;
21     }
22 }
23 }

```

The source code can be found in `src/image.cu` and the script used for the visualization was done with Matlab (more details in: `src/realistic_simulation.m`)

The visual outcome of this configuration is discussed in Subsection 4.1.

Execution

The core of the simulation is inside the `step_kernel()` function, which implements the approximation described in formula (2), shown below.

```
1 void step_kernel_mod(int ni, int nj, float fact, float* temp_in,
2   float* temp_out)
3 {
4   int i00, im10, ip10, i0m1, i0p1;
5   float d2tdx2, d2tdy2;
6
7   // loop over all points in domain (except boundary)
8   for ( int j=1; j < nj-1; j++ ) {
9     for ( int i=1; i < ni-1; i++ ) {
10      // find indices into linear memory
11      // for central point and neighbours
12      i00 = I2D(ni, i, j);
13      im10 = I2D(ni, i-1, j);
14      ip10 = I2D(ni, i+1, j);
15      i0m1 = I2D(ni, i, j-1);
16      i0p1 = I2D(ni, i, j+1);
17
18      // evaluate derivatives
19      d2tdx2 = temp_in[im10]-2*temp_in[i00]+temp_in[ip10];
20      d2tdy2 = temp_in[i0m1]-2*temp_in[i00]+temp_in[i0p1];
21
22      // update temperatures
23      temp_out[i00] = temp_in[i00]+fact*(d2tdx2 + d2tdy2);
24    }
25  }
```

Note: The formula corresponds (2) to lines 18–23 in the above code.

Verification

After all timesteps, `temp1_ref` holds the CPU reference results, while `temp1` holds the modified version results. If the error exceed a threshold (e.g. 0.0005), the simulation is considered correct.

```
1 float maxError = 0;
2
3 for( int i = 0; i < ni*nj; ++i ) {
4   if (abs(temp1[i]-temp1_ref[i]) > maxError) { maxError = abs(
5     temp1[i]-temp1_ref[i]); }
6 }
7
8 // Check and see if our maxError is greater than an error bound
9 if (maxError > 0.0005f)
10   printf("The error is NOT within acceptable bounds.");
11 else
12   printf("The error is within acceptable bounds");
```

2.2 Project Structure

The `homework2` project is organized into four main directories:

- **build**: contains the compiled executables.
- **reports**: stores the reports generated with the `-qopt-report` flag.
- **src**: includes the source code files.
- **metrics**: scripts that run the executables in **build** and save performance metrics.

2.3 Profiling

For performance profiling, I chose to use the university workstation, using Intel’s profiling tools: **VTune** and **Advisor**. The code was compiled using the Intel Compiler (**ICX**) with the following flags: `-O3 -g -xHost` to enable high optimization, generate debug information, and target the host CPU architecture.

Simulation Parameters

The analysis was conducted considering data size resulting in a sequential execution time of ~ 30 seconds, by setting:

- $nstep = 200$
- $ni = 20'000$
- $nj = 20'000$
- $size = ni \times nj = 400'000'000$

Compiler Reports

In addition to runtime profiling, I also examined the compiler optimization reports generated with the flag `-qopt-report`. These reports provide insights on which loops and functions the compiler was able to optimize or vectorize.

Advisor Roofline Analysis

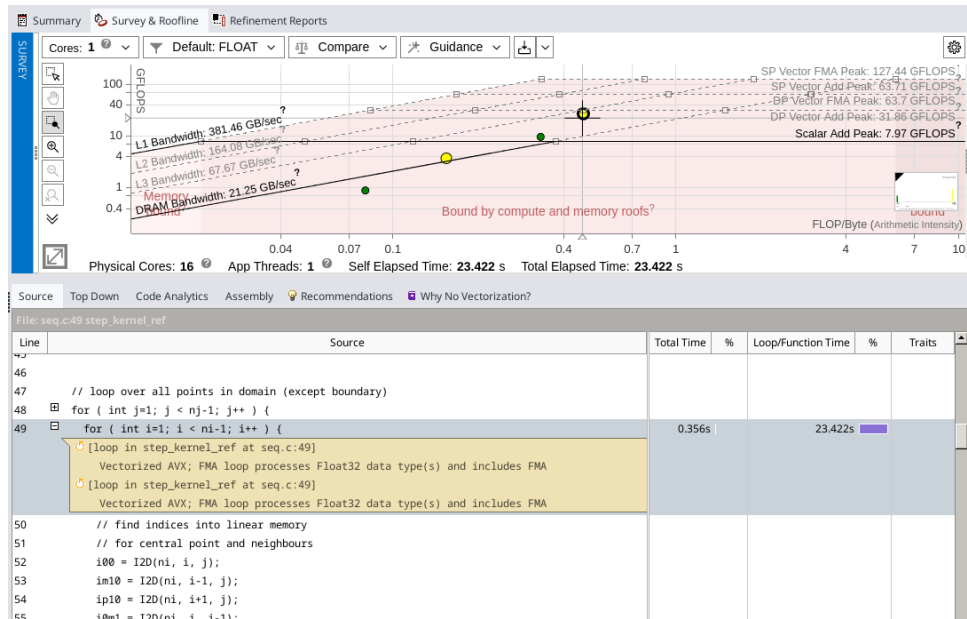


Figure 2: Roofline analysis of the sequential baseline code.

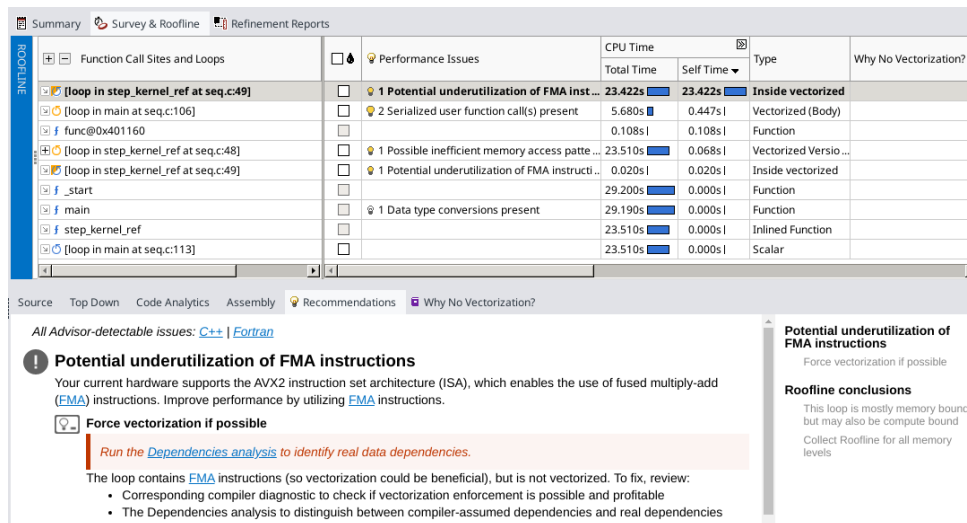


Figure 3: Advisor Survey.

TODO: aggiungere una breve descrizione di queste due immagini

3 Parallelization

3.1 Sequential section

Ho deciso di eseguire i test sul mio PC personale, piuttosto che sulla workstation dell'università o su Google Colab, per evitare limitazioni dovute a sessioni temporanee e risorse limitate. In questo modo ho avuto maggiore controllo sul processo di compilazione e sull'esecuzione dei test.

I parametri della simulazione utilizzati sono:

- $nstep = 200$
- $ni = 15000$
- $nj = 15000$
- $size = ni \times nj = 900\,000\,000$

Ho compilato il codice sequenziale presente in `src/heat.c` using the following options:

```
gcc -O3 src/heat.c -o build/heat
```

Il tempo di esecuzione migliore ottenuto TODO:(XX secs) costituisce il riferimento sequenziale per il calcolo dello speedup nella versione GPU.

3.2 CUDA

3.2.1 Code

La parallelizzazione è stata realizzata utilizzando CUDA per eseguire il calcolo dei passi temporali su GPU. Il kernel principale è `step_kernel_mod`, che replica il comportamento della funzione sequenziale `step_kernel_ref`.

Struttura del kernel:

- Ogni thread gestisce un punto della griglia 2D.
- Gli indici globali i e j vengono calcolati a partire dai `threadIdx` e `blockIdx`.

Gestione della memoria:

- Allocazione della memoria host e device.
- Copia dei dati iniziali dal host alla device.
- Copia dei risultati dalla device al host al termine della simulazione.

Controllo errori:

- Dopo ogni chiamata al kernel viene eseguito `cudaGetLastError()`.
- Viene calcolato l'errore massimo rispetto al riferimento CPU per verificare la correttezza.

3.2.2 Compilation

La compilazione CUDA è stata effettuata con il comando:

```
nvcc -O3 -arch=sm_50 cuda.cu -o cuda.exe -DDEBUG=0
```

Significato dei flag:

- `-O3`: ottimizzazione massima.
- `-arch=sm_50`: architettura target della GPU.
- `-DDEBUG=0`: disabilita il debug per migliorare le performance.

3.2.3 Execution

Per il kernel CUDA, la scelta di `threadsPerBlock` e `numBlocks` è stata la seguente:

```
1   dim3 threadsPerBlock(16, 16);  
2   dim3 numBlocks((ni + 15)/16, (nj + 15)/16);
```

Ogni blocco di 16x16 thread calcola i punti della griglia, mentre i blocchi sono organizzati per coprire l'intera matrice.

La temporizzazione è stata eseguita tramite `cudaEvent_t` per misurare il tempo del kernel GPU.

3.2.4 Results

TODO: aggiungere nvprof

I risultati della simulazione GPU sono stati confrontati con la versione CPU:

- Il tempo totale GPU è stato misurato in millisecondi.
- Lo speedup è calcolato come:

$$\text{speedup} = \frac{T_{\text{CPU}}}{T_{\text{GPU}}}$$

- L'errore massimo tra GPU e CPU è stato calcolato per verificare la correttezza dei risultati. Nella simulazione corrente, il massimo errore osservato era entro i limiti accettabili (0.0005).

I risultati confermano che la parallelizzazione con CUDA riduce significativamente i tempi di esecuzione rispetto alla versione sequenziale, mantenendo la precisione numerica.

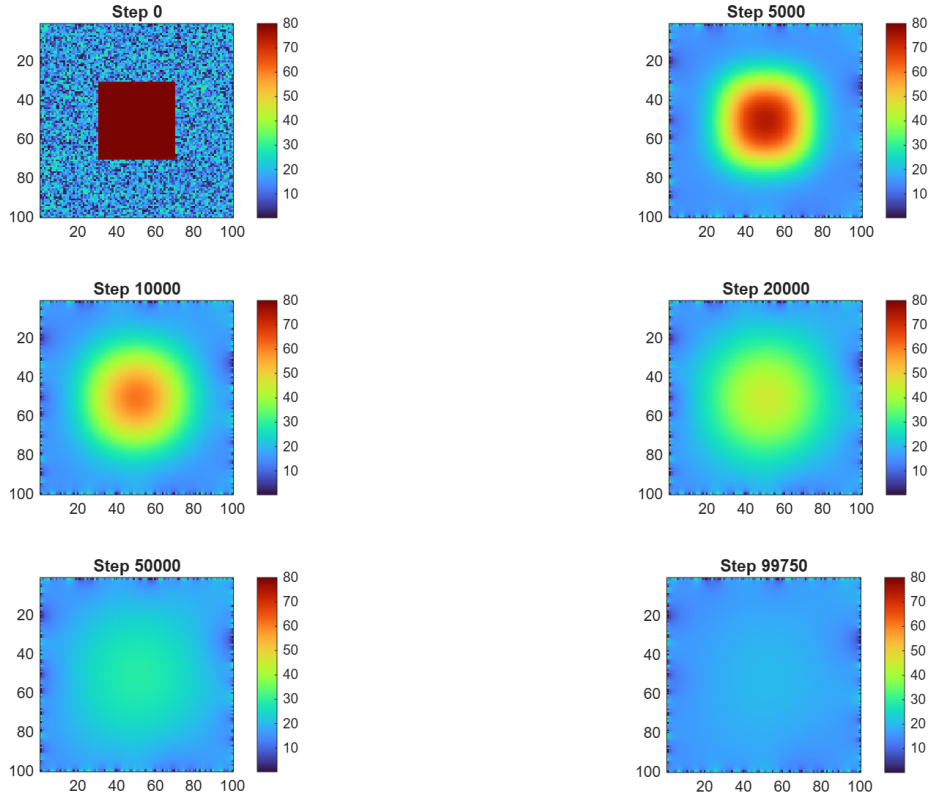
4 Conclusions

4.1 Realistic System Simulation

A seguito delle analisi descritte nel paragrafo precedente, ho voluto integrare il progetto con una simulazione grafica reale, per dimostrare che la simulazione è corretta.

I parametri della simulazione reale utilizzati sono:

- $nstep = 100'000$
- $step = 250$
- $ni = 100$
- $nj = 100$
- $tfac = 0.01f$
- $size = ni \times nj = 10'000$



4.2 Considerations

La parallelizzazione tramite GPU ha permesso di ottenere uno speedup significativo, dimostrando l'efficacia dell'approccio rispetto alla CPU. Eventuali ottimizzazioni future potrebbero includere:

- Uso della memoria condivisa per ridurre gli accessi globali.
- Tuning della dimensione dei blocchi e dei grid.
- Pipeline di esecuzione per ridurre i tempi di trasferimento host-device.